LAB MANUAL

Charotar University of Science and Technology (CHARUSAT)

Chandubhai S Patel Institute of Technology (CSPIT)

Smt. Kundanben Dinsha Patel Department of Information Technology

IT343 Operating System

A.Y. 2019-20

Practical-1

Aim: Study Practical

A. UNIX Architecture
B. Types of OS
C. Flavors of LINUX

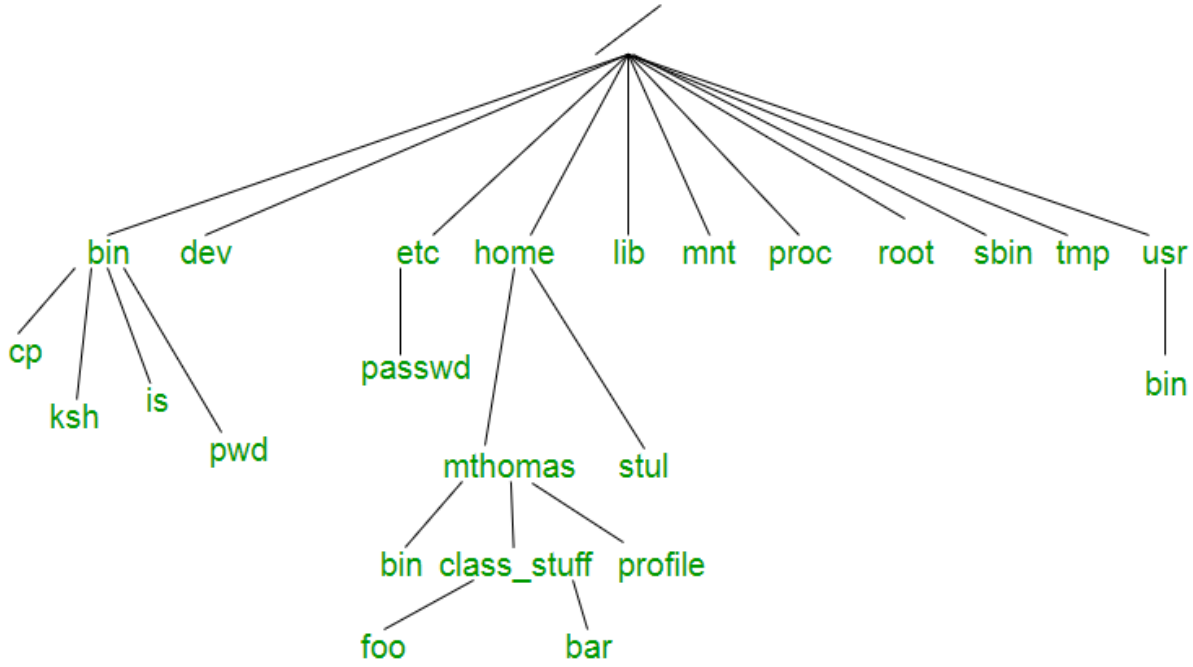
Theory:

UNIX Architecture:

"On a UNIX system, everything is a file; if something is not a file, it is a process."

This statement is true because there are special files that are more than just files (named pipes and sockets, for instance), but to keep things simple, saying that everything is a file is an acceptable generalization. A Linux system, just like UNIX, makes no difference between a file and a directory, since a directory is just a file containing names of other files. Programs, services, texts, images, and so forth, are all files. Input and output devices, and generally all devices, are considered to be files, according to the system.

In order to manage all those files in an orderly fashion, man likes to think of them in an ordered tree-like structure on the hard disk, as we know from MS-DOS (Disk Operating System) for instance. The large branches contain more branches, and the branches at the end contain the tree's leaves or normal files. For now, we will use this image of the tree, but we will find out later why this is not a fully accurate image.



Types of OS:

- Batch operating system
- Time-sharing Operating system
- Distributed operating system
- Network operating system
- Real-time operating system

Flavors of LINUX:

- **Non-Unix Operating Systems**
  - DOS?
  - Windows 3.11, 95, 98, NT, 2000
  - MacOS (Apple)
- **Unix Varieties**
  - Sun Solaris
  - Digital Unix
  - IBM AIX
  - HP-UX
  - SCO Unix
- **Linux** (the **Free** Choice!)

Practical-2

Aim: Study the following Unix commands with option:

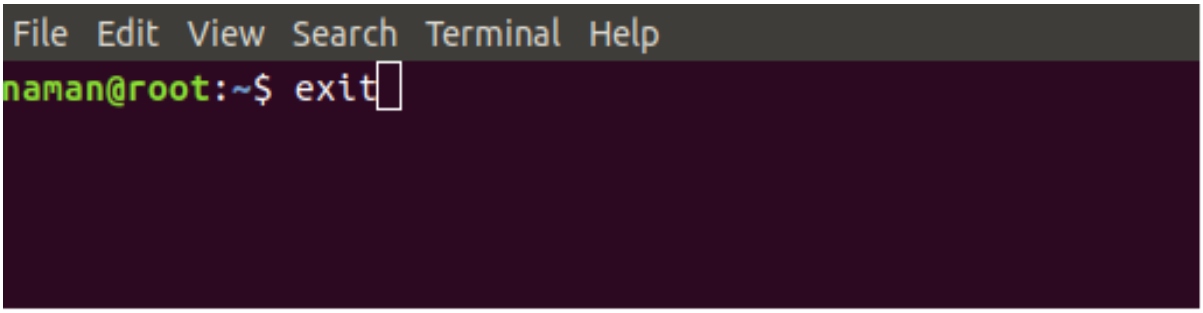
User Access:	login, logout, passwd, exit
Help:	man, help
Directory:	mkdir, rmdir, cd, pwd, ls
Editor:	vi, gedit, ed, sed
File Handling / Text Processing:	cp, mv, rm, sort, cat, pg, file, find, more, cmp, diff, comm, head, tail, cut, grep, touch, tr, uniq
Security and Protection:	chmod, chown, chgrp, newgrp
Information:	learn, man, who, date, cal, tty, calendar, time, bc, whoami, which, hostname, history, wc
System Administrator:	su or root, date, fsck, init 2, wall, shut down, mkfs, mount, unmount, dump, restor, tar, adduser, rmuser
Terminal:	echo, printf, clear
Process:	ps, kill, exec
I/O Redirection (<, >, >>), **Pipe**(|), *, gcc	

User Access Commands:

- **login:** The login program is used to establish a new session with the system. It is normally invoked automatically by responding to the "login:" prompt on the user's terminal.

- **logout:** logout command allows you to programmatically logout from your session. causes the session manager to take the requested action immediately.

- **exit**: The exit command in Linux is used to exit the shell where it is currently running.

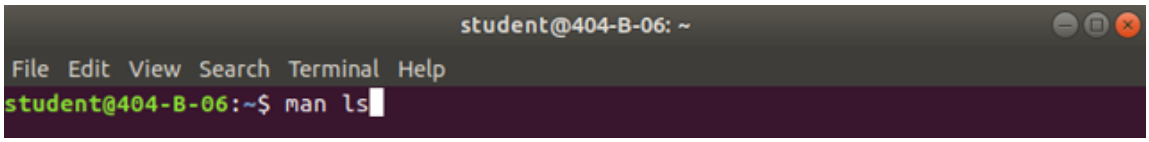


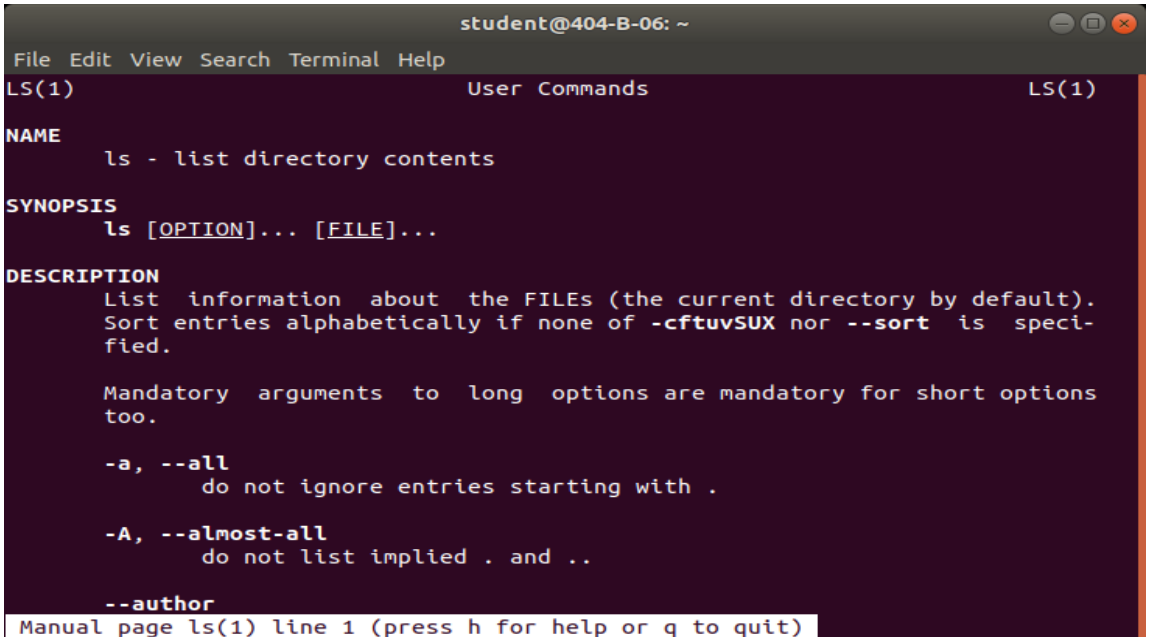
- **passwd:** The passwd command is used to change the password of a user account. A normal user can run passwd to change their own password, and a system administrator (the superuser) can use passwd to change another user's password, or define how that account's password can be used or changed.

```
ubuntu@ubuntu:~$ sudo passwd smit
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
ubuntu@ubuntu:~$ sudo passwd -l smit
passwd: password expiry information changed.
ubuntu@ubuntu:~$ sudo passwd -u smit
passwd: password expiry information changed.
ubuntu@ubuntu:~$ sudo passwd smit
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
ubuntu@ubuntu:~$
```

Help Commands:

- **man:** man is the system's manual viewer; it can be used to display manual pages, scroll up and down, search for occurrences of specific text, and other useful functions.





Directory Commands:

- **mkdir:** create new directory
- **cd:** to change the directory
- **rm:** to remove the file
- **rmdir:** to remove the directory
- **pwd:** present working directory

```
codebind@codebind-VirtualBox: ~
File  Edit  View  Search  Terminal  Help
codebind@codebind-VirtualBox:~$ pwd
/home/codebind
codebind@codebind-VirtualBox:~$ ls
Desktop     Downloads          Music      Public      Videos
Documents   examples.desktop   Pictures   Templates
codebind@codebind-VirtualBox:~$ mkdir OS
codebind@codebind-VirtualBox:~$ ls
Desktop     Downloads          Music   Pictures   Templates
Documents   examples.desktop   OS      Public     Videos
codebind@codebind-VirtualBox:~$ cd OS
codebind@codebind-VirtualBox:~/OS$ pwd
/home/codebind/OS
codebind@codebind-VirtualBox:~/OS$ cd ..
codebind@codebind-VirtualBox:~$ ls
Desktop     Downloads          Music   Pictures   Templates
Documents   examples.desktop   OS      Public     Videos
codebind@codebind-VirtualBox:~$
codebind@codebind-VirtualBox:~$ rmdir OS
codebind@codebind-VirtualBox:~$ ls
Desktop     Downloads          Music      Public      Videos
Documents   examples.desktop   Pictures   Templates
codebind@codebind-VirtualBox:~$
```

File Handling/Text Processing Commands:

- **cat:** It has three related functions with regard to text files: displaying them, combining copies of them and creating new ones.

```
student@404-B-06:~$ cat > Smit
Hello
I m Smit Ladani
Id::16IT043^Z
[1]+  Stopped                 cat > Smit
student@404-B-06:~$ ls
16IT043  Documents  examples.desktop  Pictures  Smit       Videos
Desktop  Downloads  Music             Public    Templates
student@404-B-06:~$ cat Smit
Hello
I m Smit Ladani
student@404-B-06:~$
```

- **touch:** to create blank file

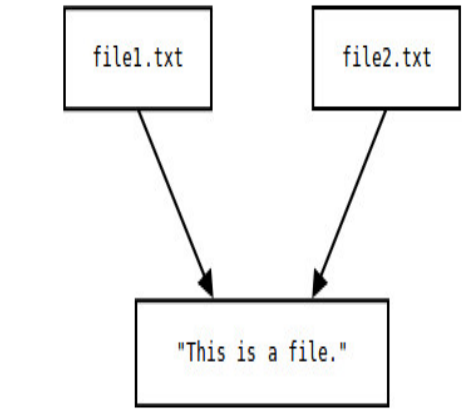
```
student@404-B-16:~$ touch -a s1
student@404-B-16:~$ touch -m s2
student@404-B-16:~$ touch s3 s4
student@404-B-16:~$ ls
16IT043  Desktop    Downloads          Music     Public  s2  s4         Videos
bin      Documents  examples.desktop   Pictures  s1      s3  Templates  xzc
student@404-B-16:~$
```

- **ls:** listing of files and folders in current directory

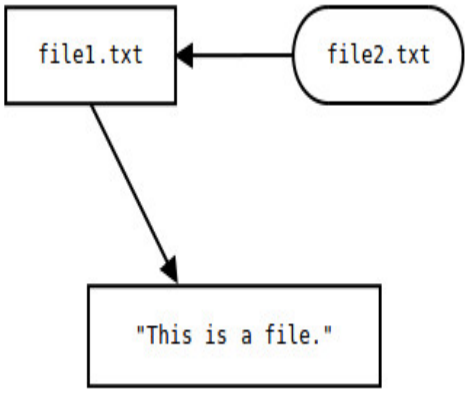
```
student@404-B-06:~$ ls
Desktop    Downloads         Music     Public     Videos
Documents  examples.desktop  Pictures  Templates
student@404-B-06:~$ ls -l
total 44
drwxr-xr-x 2 student student 4096 Jun  9 22:10 Desktop
drwxr-xr-x 2 student student 4096 Jun  9 22:10 Documents
drwxr-xr-x 2 student student 4096 Jun 10 02:28 Downloads
-rw-r--r-- 1 student student 8980 Jun  9 16:33 examples.desktop
drwxr-xr-x 2 student student 4096 Jun  9 22:10 Music
drwxr-xr-x 2 student student 4096 Jun  9 22:10 Pictures
drwxr-xr-x 2 student student 4096 Jun  9 22:10 Public
drwxr-xr-x 2 student student 4096 Jun  9 22:10 Templates
drwxr-xr-x 2 student student 4096 Jun  9 22:10 Videos
student@404-B-06:~$ ls -s
total 44
 4 Desktop     4 Downloads          4 Music      4 Public      4 Videos
 4 Documents  12 examples.desktop   4 Pictures   4 Templates
student@404-B-06:~$
```

- **ln:** The general form of the link command is: "link file_name linkname". Our first argument is the name of the file whose data we're linking to; the second argument is the name of the new link we're creating.

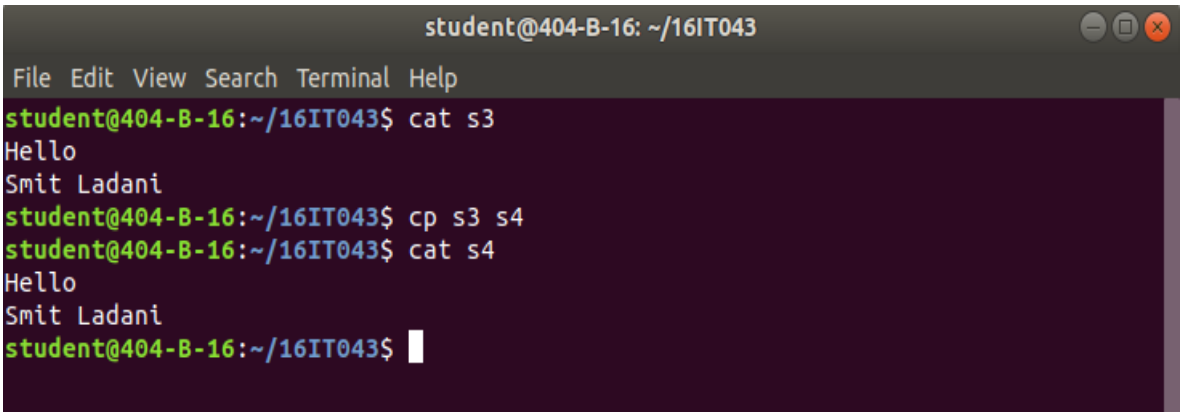
```
ubuntu@ubuntu:~$ cat file1.txt
Hello
Smit Ladani
16IT043
ubuntu@ubuntu:~$ ln file1.txt file2.txt
ubuntu@ubuntu:~$ cat file2.txt
Hello
Smit Ladani
16IT043
ubuntu@ubuntu:~$ rm file1.txt
ubuntu@ubuntu:~$ cat file2.txt
Hello
Smit Ladani
16IT043
ubuntu@ubuntu:~$
```



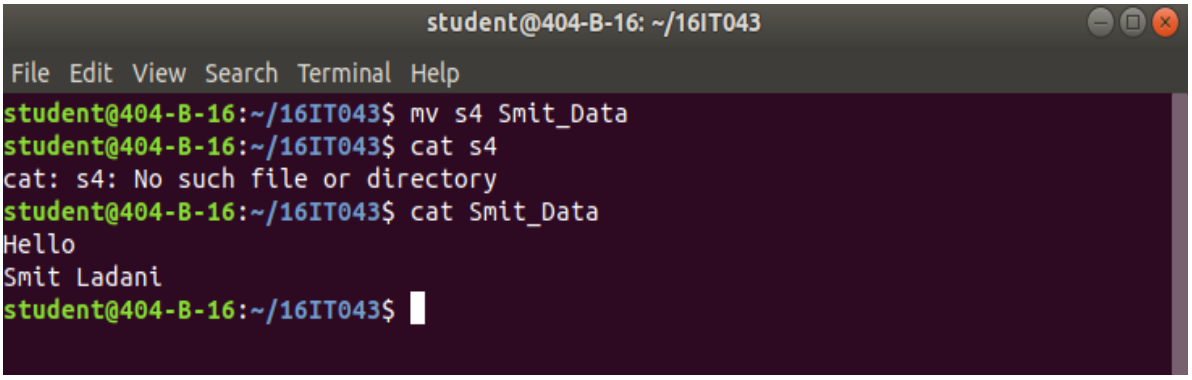

```
ubuntu@ubuntu:~$ cat file1.txt
Hello
Smit Ladani
16IT043
ubuntu@ubuntu:~$ ln -s file1.txt file2.txt
ubuntu@ubuntu:~$ cat file2.txt
Hello
Smit Ladani
16IT043
ubuntu@ubuntu:~$ rm file1.txt
ubuntu@ubuntu:~$ cat file2.txt
cat: file2.txt: No such file or directory
ubuntu@ubuntu:~$
```



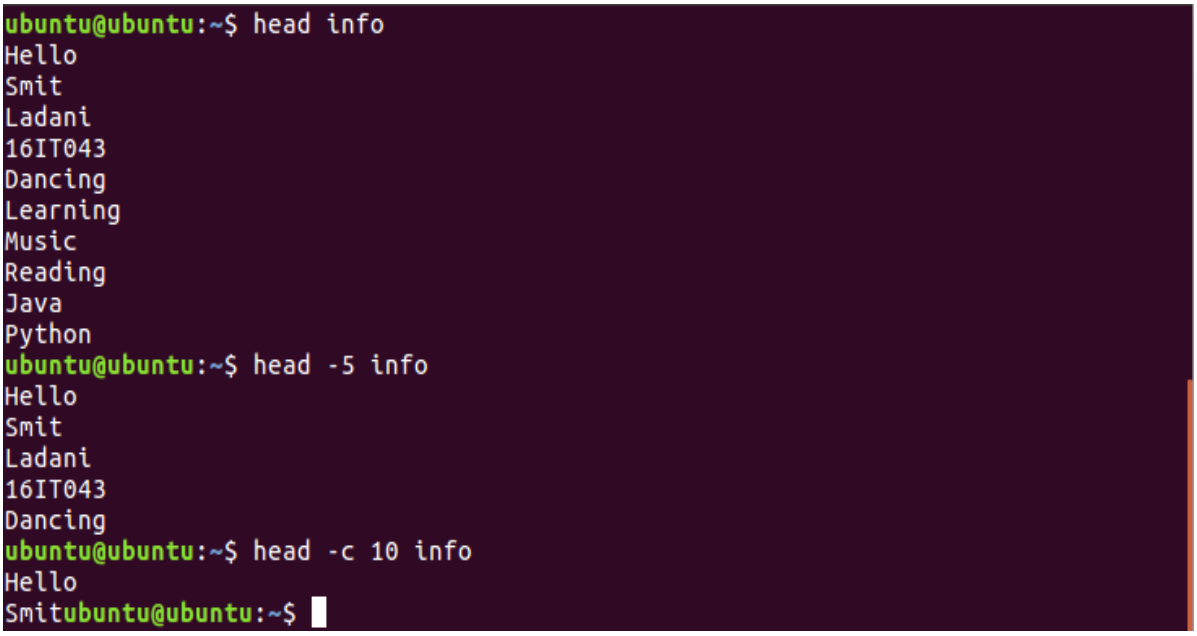
- **cp:** to copy the file1 into file2



- **mv:** to rename the file1 to file2



- **head:** head, by default, prints the first 10 lines of each FILE to standard output. With more than one FILE, it precedes each set of output with a header identifying the file name.



- **tail:** tail outputs the last part, or "tail", of files. It can also monitor new information written to the file in real time, displaying the newest entries in a system log.

```
ubuntu@ubuntu:~$ tail info
Hello
Smit
Ladani
16IT043
Dancing
Learning
Music
Reading
Java
Python
ubuntu@ubuntu:~$ tail -3 info
Reading
Java
Python
ubuntu@ubuntu:~$ tail -c 15 info
ng
Java
Python
ubuntu@ubuntu:~$
```

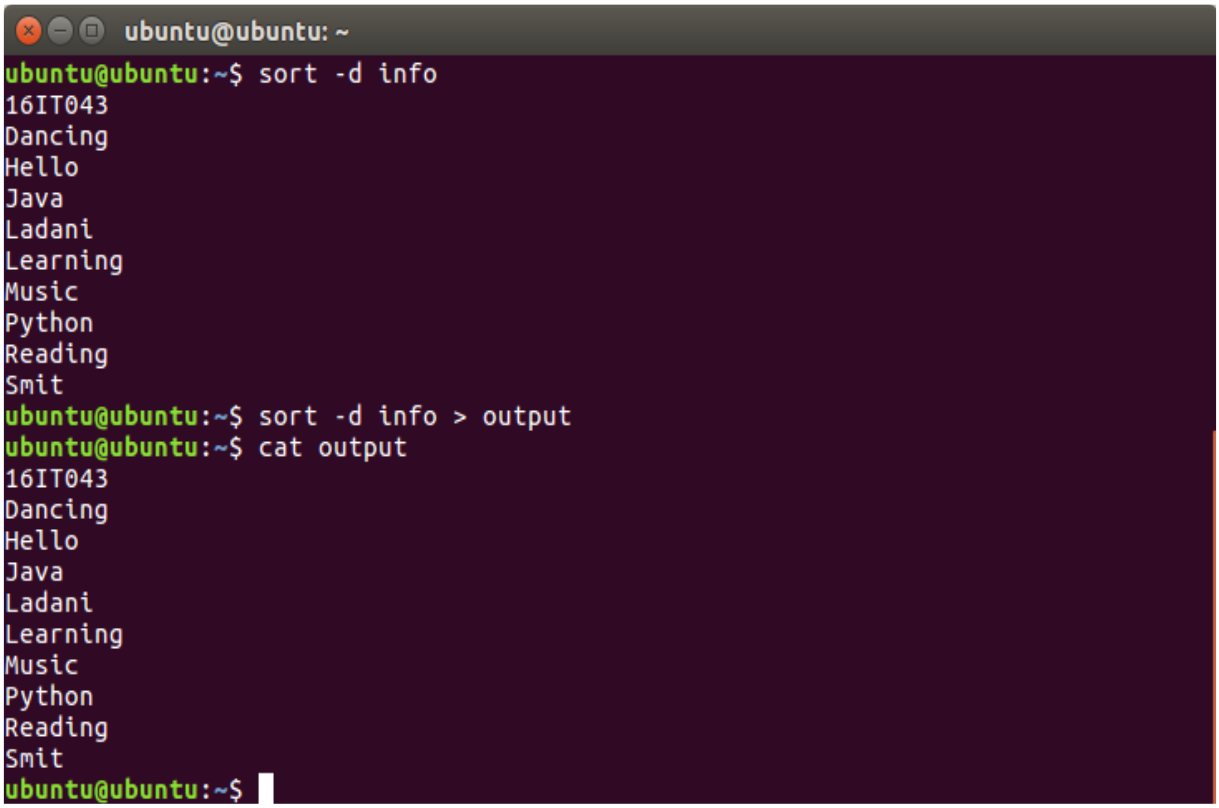
- **cut:** Remove or "cut out" sections of each line of a file or files.

```
ubuntu@ubuntu:~$ cut -b 1-3 info
Hel
Smi
Lad
16I
Dan
Lea
Mus
Rea
Jav
Pyt
ubuntu@ubuntu:~$ cut -c 4-6 info
lo
t
ani
T04
cin
rni
ic
din
a
hon
ubuntu@ubuntu:~$
```

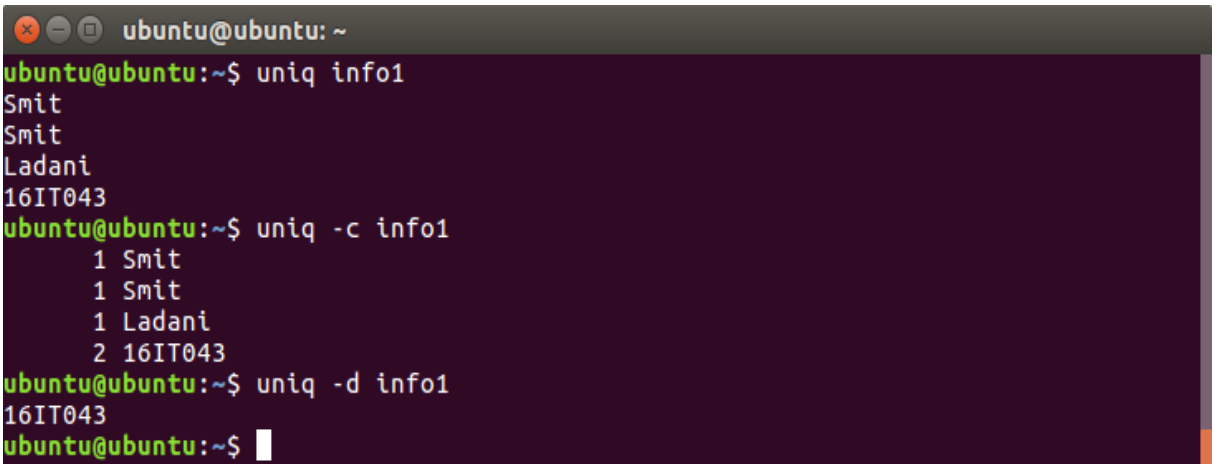
- **paste:** The paste command displays the corresponding lines of multiple files side-by-side.

```
ubuntu@ubuntu: ~
ubuntu@ubuntu:~$ paste info info1
Hello    Smit
Smit     16IT043
Ladani   Ladani
16IT043 Hello World
Dancing
Learning
Music
Reading
Java
Python
ubuntu@ubuntu:~$
```

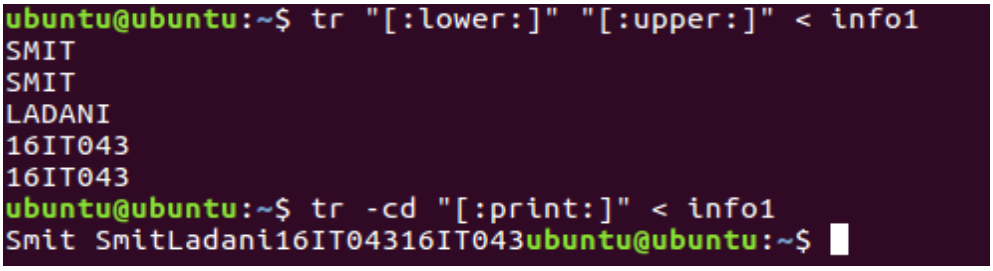
- **sort:** sort sorts the contents of a text file, line by line.



- **uniq:** uniq reports or filters out repeated lines in a file.



- **tr:** The tr command automatically translates (substitutes, or maps) one set of characters to another.



- **grep:** grep, which stands for "global regular expression print," processes text line by line and prints any lines which match a specified pattern.

```
ubuntu@ubuntu:~$ grep -w "Smit" info
Smit Ladani is here
Smit
ubuntu@ubuntu:~$ grep -cw "Smit" info
2
ubuntu@ubuntu:~$ grep -cvw "Smit" info
3
ubuntu@ubuntu:~$ cat info
Hello
Smit Ladani is here
Smit
Ladani
16IT043
ubuntu@ubuntu:~$
```

- **cmp:** cmp is used to compare two files byte by byte. If a difference is found, it reports the byte and line number where the first difference is found. If no differences are found, by default, cmp returns no output.

```
ubuntu@ubuntu:~$ cmp info info1
info info1 differ: byte 32, line 4
ubuntu@ubuntu:~$ cmp -l  info info1
32 114  61
33 141  66
34 144 111
35 141 124
36 156  60
37 151  64
38  12  63
39  61  12
40  66 114
41 111 141
42 124 144
43  60 141
44  64 156
45  63 151
ubuntu@ubuntu:~$
```

Security and Protection Commands:

- **chmod:** chmod is used to change the permissions of files or directories.

```
-rw-rw-r-- 1 ubuntu ubuntu 46 Aug  2 15:16 info
-rw-rw-r-- 1 ubuntu ubuntu 46 Aug  2 15:21 info1
drwxr-xr-x 2 ubuntu ubuntu 40 Aug  2 15:06 Music
drwxr-xr-x 2 ubuntu ubuntu 60 Aug  2 15:23 Pictures
drwxr-xr-x 2 ubuntu ubuntu 40 Aug  2 15:06 Public
drwxr-xr-x 2 ubuntu ubuntu 40 Aug  2 15:06 Templates
drwxr-xr-x 2 ubuntu ubuntu 40 Aug  2 15:06 Videos
ubuntu@ubuntu:~$ chmod u=r,g=r,o=r info1
ubuntu@ubuntu:~$ ls -l
total 8
drwxr-xr-x 2 ubuntu ubuntu 80 Aug  2 15:06 Desktop
drwxr-xr-x 2 ubuntu ubuntu 40 Aug  2 15:06 Documents
drwxr-xr-x 2 ubuntu ubuntu 40 Aug  2 15:06 Downloads
-rw-rw-r-- 1 ubuntu ubuntu 46 Aug  2 15:16 info
-r--r--r-- 1 ubuntu ubuntu 46 Aug  2 15:21 info1
```

Information Commands:

- **logname:** Returns the name of the currently logged in user.
- **uname:** Print information about the current system.

```
ubuntu@ubuntu:~$ logname
logname: no login name
ubuntu@ubuntu:~$ uname
Linux
ubuntu@ubuntu:~$
```

- **who:** The who command prints information about all users who are currently logged in.

```
ubuntu@ubuntu: ~
ubuntu@ubuntu:~$ who
ubuntu   tty7         2018-08-02 18:09 (:0)
ubuntu   pts/2        2018-08-02 13:04
ubuntu@ubuntu:~$ who -b
         system boot  2018-08-02 18:09
ubuntu@ubuntu:~$ who -d
ubuntu@ubuntu:~$ who -l
LOGIN    tty1         2018-08-02 18:09              1853 id=tty1
ubuntu@ubuntu:~$
```

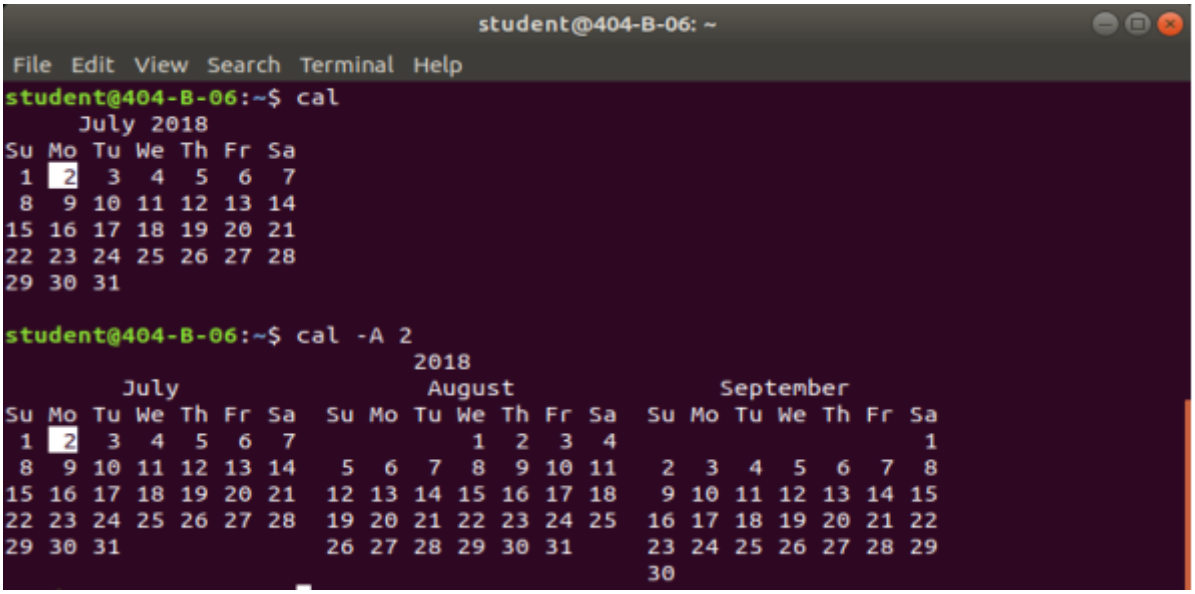
- **whoami:** This command prints the username associated with the current effective user ID.
- **tty:** Print the file name of the terminal connected to standard input.

```
ubuntu@ubuntu: ~
ubuntu@ubuntu:~$ whoami
ubuntu
ubuntu@ubuntu:~$ tty
/dev/pts/2
ubuntu@ubuntu:~$ tty -s
ubuntu@ubuntu:~$
```

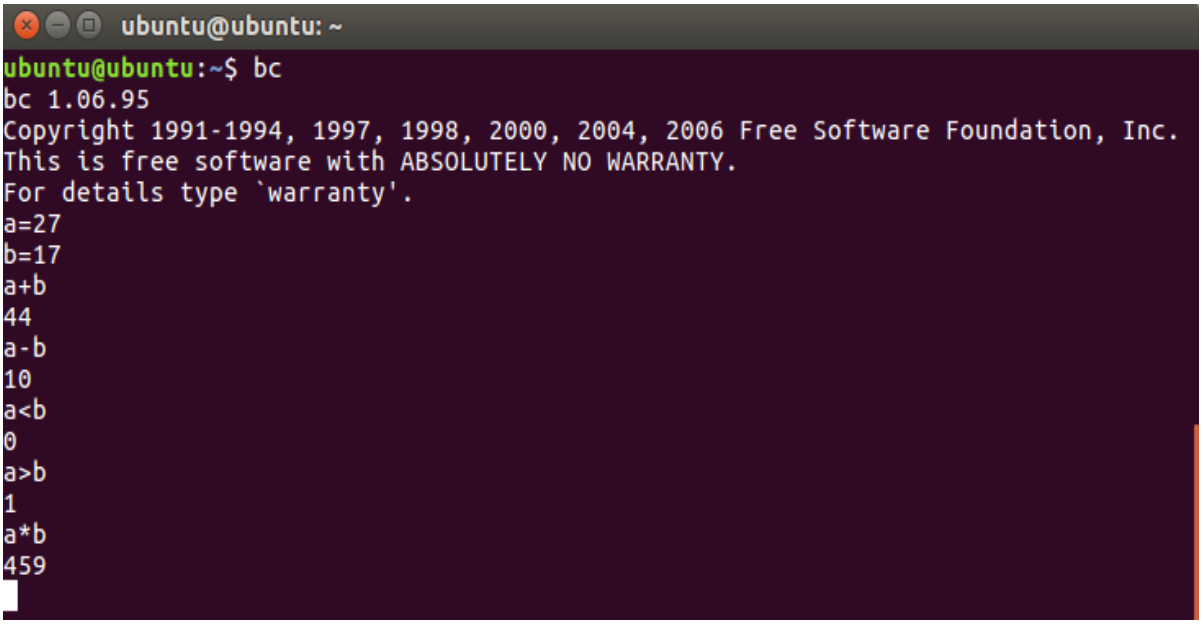
- **date:** current system date and time

```
student@404-B-06: ~
File  Edit  View  Search  Terminal  Help
student@404-B-06:~$ date
Mon Jul  2 13:26:31 IST 2018
student@404-B-06:~$ date -u
Mon Jul  2 07:56:35 UTC 2018
student@404-B-06:~$ date -R
Mon, 02 Jul 2018 13:26:41 +0530
student@404-B-06:~$
```

- **cal:** calender command



- **bc:** bc is an arbitrary-precision language for performing math calculations.



- **echo:** to print on the screen

- **expr:** expr evaluates arguments as an expression.

```
student@404-B-16:~/16IT043$ who
student  :0           2018-07-09 17:38 (:0)
student@404-B-16:~/16IT043$ tty
/dev/pts/1
student@404-B-16:~/16IT043$ expr 10 + 10
20
student@404-B-16:~/16IT043$ expr 10+10
10+10
student@404-B-16:~/16IT043$ expr 10 \* 10
100
student@404-B-16:~/16IT043$ expr 10 - 10
0
student@404-B-16:~/16IT043$ expr 10 / 5
2
student@404-B-16:~/16IT043$
```

Questions:

1. Study the Bash shell, Bourne shell and C shell in Linux operating system.
2. Study the basic environment variables: PATH, USER, HOME, SHELL. Create your own environment variable.

Practical-3

Aim:

1. Write a script called hello which outputs the following:
   - your username
   - the time and date
   - who is logged on?
   - Also output a line of asterisks (**********) after each section.
2. Write a shell program to find the largest integer among the three integers given as arguments.
3. Write a shell script to simulate a simple calculator.
4. Write a shell script to sort the number in ascending order and also calculate the shell script run time. (Using array).

Source Code:

1. Unix shell program to print out username, time and date.

```
echo "************************************************************"
echo Username:: `uname`
echo "************************************************************"
echo Date:: `date`
echo "************************************************************"
echo Logged On:: `whoami`
echo "************************************************************"
```

Output:

```
ubuntu@ubuntu:~/16IT043$ sh hello.sh
********************************************************************************
Username:: Linux
********************************************************************************
Date:: Mon Aug 27 09:22:30 UTC 2018
********************************************************************************
Logged On:: ubuntu
********************************************************************************
```

2. Unix shell program to find the largest integer among the three integers given as arguments.

```
n1=$1
n2=$2
n3=$3

if [ $n1 -ge $n2 ]
then
   if [ $n1 -ge $n3 ]
   then
         echo $n1 is maximum
   else
         echo $n3 is maximum
   fi
else
```

```
if [ $n2 -ge $n3 ]
   then
         echo $n2 is maximum
   else
         echo $n3 is maximum
   fi
fi
```

Output:

```
ubuntu@ubuntu:~/16IT043$ sh prac2.g.sh 27 10 17
27 is maximum
ubuntu@ubuntu:~/16IT043$
```

3. Unix shell script to simulate a simple calculator.

```
echo Welcome to Calculator
echo Enter 1st number::
read a
echo Enter 2nd number::
read b

echo 1. Addition
echo 2. Subtraction
echo 3. Multiplication
echo 4. Division
echo 5. Modulo
echo Enter your choice::
read choice

if [ $choice = "1" ]
then
   echo Addition of $a+$b=`expr $a + $b`
elif [ $choice = "2" ]
then
   echo Subtraction of $a-$b=`expr $a - $b`
elif [ $choice = "3" ]
then
   echo Multiplication of $a*$b=`expr $a \* $b`
elif [ $choice = "4" ]
then
   echo Division of $a/$b=`expr $a / $b`
elif [ $choice = "5" ]
then
   echo Modulo of $a%$b=`expr $a % $b`
else
   echo Invalid Choice
   echo Try again later
fi
```

Output:

```
ubuntu@ubuntu:~/16IT043$ sh Calculator_user.sh
Welcome to Calculator
Enter 1st number::
27
Enter 2nd number::
10
1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Modulo
Enter your choice::
1
Addition of 27+10=37
```

4. **Unix shell script to sort the number in ascending order and also calculate the shell script run time. (Using array).**

```
echo Enter filename::
read file

if [ -d $file ]
then
   echo $file is directory not a file
   exit 0
fi
   echo Sorted file content is:: `sort -n $file`
```

Practical 4

Aim:

1. Write a shell program to count the following in a text file.
   - Number of vowels in a given text file.
   - Number of blank spaces.
   - Number of characters.
   - Number of symbols.
   - Number of lines
2. Write a shell script which will take a file name from the user and finds that whether the file is there or not in a current working directory and displays the appropriate message.
3. Write a shell script which compares two files given by the user and if both files are same then delete the second one, if not then merge the two files in a new file.

Source Code:

1. Unix shell program to count number of vowels, blank spaces, characters, symbols and lines.

```
echo Enter filename::
read file

if [ -d $file ]
then
   echo $file is directory not file
   exit 0
fi

if [ -f $file ]
then
   echo Total number of vowels:: `grep -oi [aeiou] $file | wc -l`
   echo Total number of blank space:: `grep -o [[:space:]] $file | wc -l`
   echo Total number of characters:: `wc -c $file`
   echo Total number of symbols:: `grep -oi [^{a-zA-Z}{0-9}[:space:]]
   $file | wc -l`
   echo Total number of lines:: `wc -l $file`
fi
```

Output:

```
ubuntu@ubuntu:~/16IT043$ sh prac2.f.sh
Enter filename::
f1.txt
Total number of vowels:: 8
Total number of blank space:: 3
Total number of characters:: 37 f1.txt
Total number of symbols:: 4
Total number of lines:: 3 f1.txt
```

2. **Unix shell script which will take a file name from the user and finds that whether the file is there or not in a current working directory and displays the appropriate message.**

```
echo -e "Enter the name of the file : \c"
read file_name

if [ -f $file_name ]
then
   echo "$file_name is exists"
else
  echo "$file_name not exists"
fi
```

3. **Unix shell script which compares two files given by the user and if both files are same then delete the second one, if not then merge the two files in a new file.**

```
echo Enter filename1::
read file1
echo Enter filename2::
read file2

if cmp -s $file1 $file2
then
   rm $file2
else
   cat $file1 $file2 > newfile.txt
fi
```

Practical-5

Aim: Write a C program to list for every file in a directory, its inode number and file name.

Source code:

```
#include<stdlib.h>
#include<stdio.h>
#include<string.h>

void main(int argc, char *argv[])
{
      char d[50] = "";
      if(argc==2)
      {
            strcat(d,"ls ");
            strcat(d,"-i ");
            strcat(d,argv[1]);
            system(d);
      }
      else
            printf("\nInvalid No. of inputs");
}
```

Output:

```
student@404-B-16:~/16IT043/Practical-3$ gcc practical3.c -o practical3.out
student@404-B-16:~/16IT043/Practical-3$ ./practical3.out a.out
1833858 a.out
student@404-B-16:~/16IT043/Practical-3$ ./practical3.out practical3.c
1833851 practical3.c
```

Question:

1. What is command to run C program in terminal
2. What is inode number in linux.

Practical-6

Aim: Write a C program in UNIX to implement Process scheduling algorithms and compare.

A. First Come First Serve (FCFS) Scheduling
B. Shortest-Job-First (SJF) Scheduling

Source Code:

A. First Come First Serve (FCFS) Scheduling

```
#include<stdio.h>
main()
{
   int bt[20], wt[20], tat[20], i, n;
   float wtavg, tatavg;
   printf("\nEnter the number of processes -- ");
   scanf("%d", &n);
   for(i=0;i<n;i++)
   {
         printf("\nEnter Burst Time for Process %d -- ", i);
         scanf("%d", &bt[i]);
   }
   wt[0] = wtavg = 0;
   tat[0] = tatavg = bt[0];
   for(i=1;i<n;i++)
   {
         wt[i] = wt[i-1] +bt[i-1];
         tat[i] = tat[i-1] +bt[i];
          wtavg = wtavg + wt[i];
          tatavg = tatavg + tat[i];
   }
   printf("\t PROCESS \tBURST TIME \t WAITING TIME\t TURNAROUND TIME\n");
   for(i=0;i<n;i++)
   {
         printf("\n\t P%d \t\t %d \t\t %d \t\t %d",i,bt[i],wt[i],tat[i]);
   }
   printf("\nAverage Waiting Time -- %f", wtavg/n);
   printf("\nAverage Turnaround Time -- %f", tatavg/n);
}
```

1. Shortest-Job-First (SJF) Scheduling

```
#include<stdio.h>
main()
{

   int p[20], bt[20], wt[20], tat[20], i, k, n, temp;
   float wtavg, tatavg;
   printf("\nEnter the number of processes -- ");
```

```
     scanf("%d", &n);
     for(i=0;i<n;i++)
    {
          p[i]=i;
          printf("Enter Burst Time for Process %d -- ", i);
          scanf("%d", &bt[i]);
    }
    for(i=0;i<n;i++)
          for(k=i+1;k<n;k++)
                if(bt[i]>bt[k])
                {
                      temp=bt[i];
                      bt[i]=bt[k];
                      bt[k]=temp;
                      temp=p[i];
                      p[i]=p[k];
                      p[k]=temp;
                }
                wt[0] = wtavg = 0;
                tat[0] = tatavg = bt[0];
                for(i=1;i<n;i++)
                {
                      wt[i] = wt[i-1] +bt[i-1];
                      tat[i] = tat[i-1] +bt[i];
                      wtavg = wtavg + wt[i];
                      tatavg = tatavg + tat[i];
                }
    printf("\n\t PROCESS\t URST\t waiting\t TURNAROUND\n");
    for(i=0;i<n;i++)
    {
          printf("\n\tP%d\t\t%d\t\t%d\t\t%d",p[i],bt[i],wt[i],tat[i]);
    }
    printf("\nAverage Waiting Time -- %f", wtavg/n);
    printf("\nAverage Turnaround Time -- %f", tatavg/n);

}
```

Practical-7

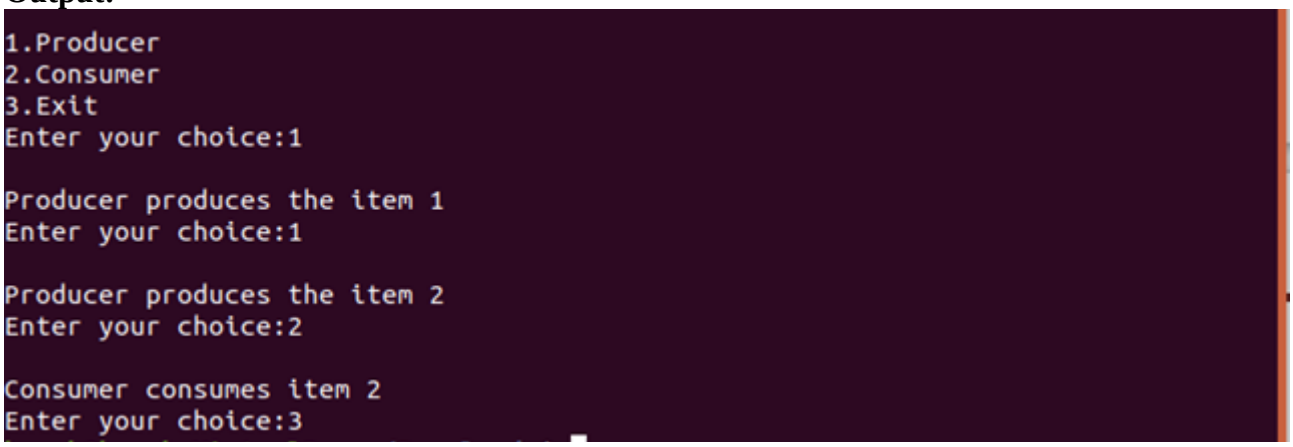
Aim: Write a C program in UNIX to implement inter process communication (IPC) using Semaphore.

Source code:

```
#include<stdio.h>
#include<stdlib.h>
int mutex=1,full=0,empty=3,x=0;

int main()
{
      int n;
      void producer();
      void consumer();
      int wait(int);
      int signal(int);
      printf("\n1.Producer\n2.Consumer\n3.Exit");
      while(1)
      {
            printf("\nEnter your choice:");
            scanf("%d",&n);
            switch(n)
            {
                  case 1: if((mutex==1)&&(empty!=0))
                             producer();
                          else
                             printf("Buffer is full!!");
                          break;
                  case 2: if((mutex==1)&&(full!=0))
                             consumer();
                          else
                             printf("Buffer is empty!!");
                          break;
                  case 3:
                          exit(0);
                          break;
            }
      }
      return 0;
}

int wait(int s)
{
      return (--s);
}

int signal(int s)
{
      return(++s);
}

void producer()
{
```

```
      mutex=wait(mutex);
      full=signal(full);
      empty=wait(empty);
      x++;
      printf("\nProducer produces the item %d",x);
      mutex=signal(mutex);
}

void consumer()
{
      mutex=wait(mutex);
      full=wait(full);
      empty=signal(empty);
      printf("\nConsumer consumes item %d",x);
      x--;
      mutex=signal(mutex);
}
```

Output:



Question:

1. What are semaphores?
2. List two ways by which process communicates with each other.

Practical-8

Aim: Write a C program in UNIX to implement Bankers algorithm for Deadlock Avoidance.

Source Code:

```
#include<stdio.h>
int isSafe(int p,int r,int allocation[p][r],int maximum[p][r],int
available[r])
{
      int need[p][r];
      //calculating need
      for(int i=0; i<p; i++)
      {
            for(int j=0; j<r; j++)
            {
                  need[i][j] = maximum[i][j] - allocation[i][j];
            }
      }

      int finish[p];
      for(int i=0; i<p; i++)
      {
            finish[i] = 0;
      }
      int finishSeq[p];
      int count = 0;
      while(count<p)
      {
            int found = 0;
            for(int i=0; i<p; i++)
            {
                  if(finish[i] == 0)
                  {
                        int flag = 1;
                        for(int j=0; j<r; j++)
                        {
                              if(need[i][j]>available[j])
                              {
                                    flag = 0;
                                    break;
                              }
                        }
                        if(flag == 1)
                        {
                              for(int j=0; j<r; j++)
```

```
                        {

                              available[j]=available[j]+allocation[i][j];
                        }
                        finish[i] = 1;
                        finishSeq[count++] = i;
                        found = 1;
                  }
            }
      }
      if(found == 0)
      {
            printf("System is not in safe state");
            return 0;
      }
  }
  printf("System is in safe state:\n");
  for(int i=0; i<p; i++)
  {
      printf("%d ",finishSeq[i]);
  }
  printf("\n");
}

void main()
{
  int p,r;
  printf("Enter no of processes: ");
  scanf("%d",&p);
  printf("Enter no of resources: ");
  scanf("%d",&r);
  int allocation[p][r];
  int maximum[p][r];
  int available[r];
  //User entry of allocated resource
  printf("Enter no of allocated resource: ");
  for(int i=0;i<p;i++)
  {
      for(int j=0;j<r;j++)
      {
            scanf("%d",&allocation[i][j]);
      }
  }
  //User entry of maximum resource
  printf("Enter no of maximum resource: ");

  for(int i=0;i<p;i++)
```

```
	{

		for(int j=0;j<r;j++)

		{
			scanf("%d",&maximum[i][j]);
		}
	}
	//User entry of available resource
	printf("Enter no of available resource: ");
	for(int i=0;i<r;i++)
	{
		scanf("%d",&available[i]);
	}
	isSafe(p,r,allocation,maximum,available);
}
```

Output:

```
student@404-b-19:~/16IT043$ gcc practical7.1.c -o practical7.1.out
student@404-b-19:~/16IT043$ ./practical7.1.out
Enter no of processes: 5
Enter no of resources: 3
Enter no of allocated resource: 0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter no of maximum resource: 7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter no of available resource: 3 3 2
System is in safe state:
1 3 4 0 2
```

Questions:

1. What are different methods for handling deadlock?
2. What is Livelock?

Practical-9

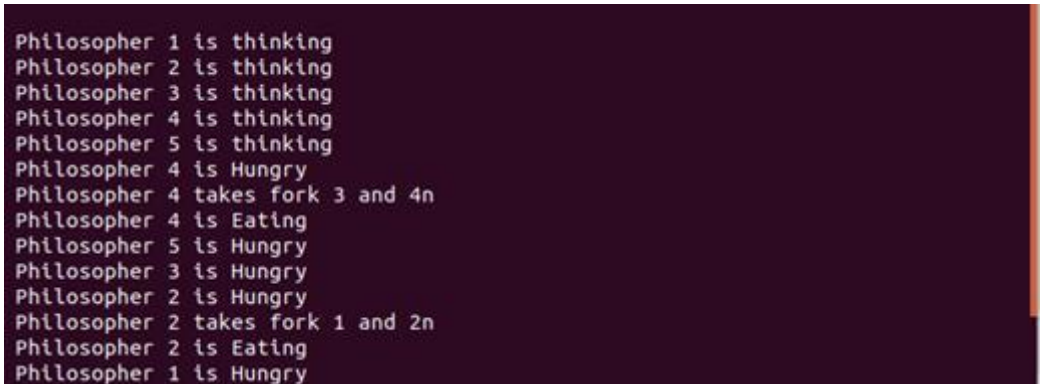
Aim: Write a C program in UNIX to solve dining philosophers' problem.

Source code:

```
#include<stdio.h>
#include<semaphore.h>
#include<pthread.h>
#define N 5
#define THINKING 0
#define HUNGRY 1
#define EATING 2
#define LEFT (ph_num+4)%N
#define RIGHT (ph_num+1)%N
sem_t mutex;
sem_t S[N];
void * philospher(void *num);
void take_fork(int);
void put_fork(int);
void test(int);
int state[N];
int phil_num[N]={0,1,2,3,4};

int main()
{
      int i;
      pthread_t thread_id[N];
      sem_init(&mutex,0,1);
      for(i=0;i<N;i++)
      sem_init(&S[i],0,0);
      for(i=0;i<N;i++)
      {
            pthread_create(&thread_id[i],NULL,philospher,&phil_num[i]);
            printf("\n Philosopher %d is thinking",i+1);
      }
      for(i=0;i<N;i++)
            pthread_join(thread_id[i],NULL);
}

void *philospher(void *num)
{
      while(1)
      {
            int *i = num;
            sleep(1);
```

```
            take_fork(*i);
            sleep(0);
            put_fork(*i);
      }
}

void take_fork(int ph_num)
{
      sem_wait(&mutex);
      state[ph_num] = HUNGRY;
      printf("\n Philosopher %d is Hungry",ph_num+1);
      test(ph_num);
      sem_post(&mutex);
      sem_wait(&S[ph_num]);
      sleep(1);
}

void test(int ph_num)
{
      if (state[ph_num] == HUNGRY && state[LEFT] != EATING && state[RIGHT]
      != EATING)
      {
            state[ph_num] = EATING;
            sleep(2);
            printf("\n Philosopher %d takes fork %d and %dn", ph_num+1,
            LEFT+1, ph_num+1);
            printf("\n Philosopher %d is Eating",ph_num+1);
            sem_post(&S[ph_num]);
      }
}

void put_fork(int ph_num)
{
      sem_wait(&mutex);
      state[ph_num] = THINKING;
      printf("\n Philosopher %d putting fork %d and %d
      down",ph_num+1,LEFT+1,ph_num+1);
      printf("\n Philosopher %d is thinking",ph_num+1);
      test(LEFT);
      test(RIGHT);
      sem_post(&mutex);
}
```

Output:



Question:

1. What is difference between thread and process?
2. How to compile a c program that uses pthread.h?

Practical-10

Aim: Write a C program in UNIX to perform Memory allocation algorithms and calculate Internal and External Fragmentation. (First Fit, Best Fit, Worst Fit).

Source Code:

1. **Worst-Fit:**

```
#include<stdio.h>
#define max 25
void main()
{
   int frag[max],b[max],f[max],i,j,nb,nf,temp;
   static int bf[max],ff[max];
   printf("\n\tMemory Management Scheme - First Fit");
   printf("\nEnter the number of blocks:");
   scanf("%d",&nb);
   printf("Enter the number of files:");
   scanf("%d",&nf);
   printf("\nEnter the size of the blocks:-\n");
   for(i=1;i<=nb;i++)
   {
         printf("Block %d:",i);
         scanf("%d",&b[i]);
   }
   printf("Enter the size of the files :-\n");
   for(i=1;i<=nf;i++)
   {
         printf("File %d:",i);
         scanf("%d",&f[i]);
   }
   for(i=1;i<=nf;i++)
   {
         for(j=1;j<=nb;j++)
         {
               if(bf[j]!=1)
               {
                     temp=b[j]-f[i];
                     if(temp>=0)
                     {
                           ff[i]=j; break;
                     }
               }
          }
         frag[i]=temp;
         bf[ff[i]]=1;
```

```
    }
    printf("\nFile_no:\tFile_size :\tBlock_no:\tBlock_size:\tFragement");
    for(i=1;i<=nf;i++)

    printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d",i,f[i],ff[i],b[ff[i]],frag[i]);
}
```

2. Best-fit:

```
#include<stdio.h>
#define max 25
void main()
{
    int frag[max],b[max],f[max],i,j,nb,nf,temp,lowest=10000;
    static int bf[max],ff[max];
    printf("\nEnter the number of blocks:");
    scanf("%d",&nb);
    printf("Enter the number of files:");
    scanf("%d",&nf);
    printf("\nEnter the size of the blocks:-\n");
    for(i=1;i<=nb;i++)
          printf("Block %d:",i);
    scanf("%d",&b[i]);
    printf("Enter the size of the files :-\n");
    for(i=1;i<=nf;i++)
    {
          printf("File %d:",i);
          scanf("%d",&f[i]);
    }
    for(i=1;i<=nf;i++)
    {
          for(j=1;j<=nb;j++)
          {
                if(bf[j]!=1)
                {
                      temp=b[j]-f[i];
                      if(temp>=0)
                             if(lowest>temp)
                            {
                                  ff[i]=j;
                                  lowest=temp;
                            }
                }
          }
          frag[i]=lowest;
          bf[ff[i]]=1;
          lowest=10000;
    }
```

```
    printf("\nFile No\tFile Size \tBlock No\tBlock Size\tFragment");
    for(i=1;i<=nf && ff[i]!=0;i++)

    printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d",i,f[i],ff[i],b[ff[i]],frag[i]);
}
```

3. First-fit:

```
#include<stdio.h>
#define max 25
void main()
 {
   int frag[max],b[max],f[max],i,j,nb,nf,temp,highest=0;
   static int bf[max],ff[max];
   clrscr();
   printf("\n\tMemory Management Scheme - Worst Fit");
   printf("\nEnter the number of blocks:");
   scanf("%d",&nb);
   printf("Enter the number of files:");
   scanf("%d",&nf);
   printf("\nEnter the size of the blocks:-\n");
   for(i=1;i<=nb;i++)
    {
         printf("Block %d:",i);
         scanf("%d",&b[i]);
   }
   printf("Enter the size of the files :-\n");
   for(i=1;i<=nf;i++)
   {
         printf("File %d:",i);
         scanf("%d",&f[i]);
   }
    for(i=1;i<=nf;i++)
   {
         for(j=1;j<=nb;j++)
         {
               if(bf[j]!=1) //if bf[j] is not allocated
               {
                     temp=b[j]-f[i];
                     if(temp>=0)
                          if(highest<temp)
                          {
                                ff[i]=j;
                                highest=temp;
                          }
               }
         }
         frag[i]=highest;
```

```
            bf[ff[i]]=1;
            highest=0;
        }
        printf("\nFile_no:\tFile_size :\tBlock_no:\tBlock_size:\tFragement");
        for(i=1;i<=nf;i++)
         printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d",i,f[i],ff[i],b[ff[i]],frag[i]);
    }
```